



Composer & Packagist.org

Gestion des dependances en PHP

Installation * Configuration * Commandes * Packages

2012

Annee de
creation

350k+

Packages
disponibles

2B+

Downloads
par mois

PHP

Langage
cible

Tout ce qu'il faut savoir pour installer, configurer
et utiliser Composer avec Packagist dans tes projets PHP.

Guide Pedagogique — Khadija

2025 — Developpement PHP

Table des Matieres

1	Qu'est-ce que Composer ?	p.3
2	Qu'est-ce que Packagist.org ?	p.3
3	Installer Composer	p.4
4	Comprendre composer.json	p.5
5	Les commandes essentielles	p.6
6	Trouver et installer un package	p.7
7	Autoloading — charger ses fichiers automatiquement	p.8
8	Composer dans un projet PHP reel	p.9
9	Bonnes pratiques & erreurs courantes	p.10

1 Qu'est-ce que Composer ?

Composer est le **gestionnaire de dependances officiel de PHP**. Une dependance c'est une librairie externe dont ton projet a besoin pour fonctionner. Composer te permet de les installer, les mettre a jour, et les gerer automatiquement sans copier-coller de fichiers manuellement.

■ L'analogie du supermarche

Imagine que tu veux cuisiner un gateau. Au lieu d'aller chercher chaque ingredient separement (farine, oeufs, sucre...), tu donnes ta liste a quelqu'un qui ramene tout. Composer c'est ce quelqu'un : tu listes tes librairies, il les telecharge.

■ Sans Composer vs Avec Composer

SANS : tu telecharges le ZIP de chaque librairie, tu l'extrais, tu copies les fichiers, tu geres les mises a jour a la main. AVEC : une seule commande installe tout, gere les versions et les dependances des dependances.

■ Gestion automatique des dependances

Il installe non seulement la librairie que tu veux, mais aussi toutes les librairies dont ELLE a besoin.

■ Gestion des versions

Tu peux specifier exactement quelle version tu veux : ^2.0 (compatible 2.x), ~1.5 (patch seulement), 3.1.2 (exacte).

■ Mise a jour simple

composer update met a jour toutes les dependances selon les contraintes definies dans composer.json.

■ Autoloading

Composer genere automatiquement un fichier qui charge tes classes PHP sans require() manuel.

2 Qu'est-ce que Packagist.org ?

Packagist.org est le **depot central de packages PHP**. C'est le "supermarche" dont Composer est le livreur. Quand tu fais composer require nom/package, Composer va chercher le package sur Packagist.

■ 350 000+ packages disponibles

Des librairies pour tout : envoyer des emails, manipuler des images, se connecter a des APIs, gerer des PDF, faire du cache...

- **Moteur de recherche**
Sur packagist.org tu peux chercher par nom, description, ou type de package. Chaque package affiche ses stats : downloads, popularite, version.
- **Statistiques en temps reel**
Nombre de downloads, etoiles GitHub, frequence de mise a jour. Tu peux evaluer si un package est maintenu activement.
- **Lien direct avec GitHub**
Chaque package sur Packagist pointe vers son repo GitHub. Code source, issues, documentation : tout est accessible.
- **Versionnage SemVer**
Tous les packages sur Packagist suivent SemVer. Tu vois toutes les versions disponibles et les notes de changement.

■ Packagist est le depot par DEFAUT de Composer. Il n'est pas necessaire de configurer quoi que ce soit : Composer y accede automatiquement. Tu peux aussi avoir des depots prives (ex: Satis pour les entreprises).

3

Installer Composer

Composer s'installe une seule fois sur ta machine. Ensuite il est disponible partout en ligne de commande.

Sur Windows

1

Telecharger l'installeur

Va sur getcomposer.org et telecharge Composer-Setup.exe

2

Lancer l'installeur

Suivre les etapes, il detecte automatiquement ton PHP.exe

3

Verifier l'installation

Ouvre un terminal et tape : `composer --version`

Sur Linux / macOS

BASH

```
# Telecharger et installer Composer globalement
php -r "copy('https://getcomposer.org/installer', 'composer-setup.php');"
php composer-setup.php
sudo mv composer.phar /usr/local/bin/composer

# Verifier l'installation
composer --version

# Resultat attendu : Composer version 2.x.x
```

Verifier que tout fonctionne

BASH

```
# Liste des commandes disponibles
composer

# Infos sur ton environnement PHP + Composer
composer diagnose

# Aide sur une commande specifique
composer help require
```

★ Composer nécessite PHP 7.2.5 minimum. Mais pour tes projets, utilise toujours PHP 8.1+ comme vu dans le guide PHP. Composer 2.x (la version actuelle) est beaucoup plus rapide que Composer 1.x.

4

Comprendre composer.json

Le fichier **composer.json** est le coeur de tout projet Composer. C'est lui qui decrit ton projet et liste toutes les dependances. Il se trouve toujours a la racine de ton projet.

JSON

```
{
  "name": "khadija/wallet-manager",
  "description": "Application de gestion de wallets",
  "type": "project",
  "version": "0.1.0",
  "require": {
    "php": ">=8.1",
    "vlucas/phpdotenv": "^5.5",
    "monolog/monolog": "^3.0"
  },
  "require-dev": {
    "phpunit/phpunit": "^10.0"
  },
  "autoload": {
    "psr-4": {
      "App\\": "src/"
    }
  },
  "autoload-dev": {
    "psr-4": {
      "Tests\\": "tests/"
    }
  }
}
```

Cle	Role	Exemple
name	Nom du projet (vendor/projet)	khadija/wallet-manager
description	Description courte du projet	Application de gestion...

<code>type</code>	Type : project, library, etc.	<code>project</code>
<code>require</code>	Dependances de PRODUCTION	<code>monolog/monolog: ^3.0</code>
<code>require-dev</code>	Dependances de DEVELOPPEMENT seulement	<code>phpunit/phpunit: ^10.0</code>
<code>autoload</code>	Configuration du chargement automatique	<code>psr-4: {App: src/}</code>

Les contraintes de version

Symbol e	Signification	Exemple	Accepte
<code>^</code>	Compatible avec la version mineure	<code>^2.3.0</code>	2.3.x et 2.4.x mais pas 3.0.0
<code>~</code>	Seulement les patches	<code>~2.3.0</code>	2.3.0 a 2.3.x mais pas 2.4.0
<code>*</code>	Toutes les versions	<code>2.*</code>	Toute version 2.x.x
<code>exact</code>	Version exacte uniquement	<code>2.3.1</code>	Uniquement 2.3.1
<code>>=</code>	Version minimum	<code>>=2.0</code>	2.0 et toutes versions superieures

5

Les commandes essentielles

Voici les commandes que tu utiliseras au quotidien. Elles suffisent pour 95% des situations.

Commande	Ce qu'elle fait
<code>composer init</code>	Initialiser un nouveau projet Composer (cree le composer.json interactivement)
<code>composer install</code>	Installer toutes les dependances du composer.json (1ere fois sur un projet)
<code>composer update</code>	Mettre a jour les dependances selon les contraintes de version
<code>composer require vendor/package</code>	Ajouter un nouveau package et l'ajouter au composer.json
<code>composer require --dev vendor/pkg</code>	Ajouter un package uniquement pour le developpement
<code>composer remove vendor/package</code>	Supprimer un package du projet
<code>composer dump-autoload</code>	Regenerer le fichier d'autoloading
<code>composer show</code>	Lister tous les packages installes
<code>composer outdated</code>	Voir les packages qui ont des mises a jour disponibles
<code>composer validate</code>	Verifier que le composer.json est valide

Commandes avancees utiles

BASH

```
# Installer SANS les packages de dev (pour la production)
composer install --no-dev

# Voir exactement ce que Composer va faire avant de le faire
composer update --dry-run

# Chercher un package directement depuis le terminal
composer search monolog

# Voir les infos d'un package installe
composer show monolog/monolog

# Mettre a jour Composer lui-meme
```

```
composer self-update
```

6

Trouver et installer un package

Voici le processus complet, de la recherche à l'utilisation d'un package.

1

Rechercher sur Packagist.org

Va sur packagist.org, tape ce que tu cherches. Ex: 'email' ou 'pdf' ou 'image'. Regarde le nombre de downloads et la date de dernière mise à jour.

2

Vérifier le package

Clique sur le package. Lis la description, regarde le README sur GitHub. Vérifie qu'il supporte ta version PHP (ex: php: >=8.0).

3

Installer le package

Copie la commande affichée sur Packagist et exécute-la dans ton terminal.

4

Utiliser le package

Inclure vendor/autoload.php dans ton fichier PHP, puis utiliser la classe.

Exemple concret : installer Monolog (bibliothèque de logs)

BASH

```
# Etape 1 : installer le package
composer require monolog/monolog

# Etape 2 : utiliser dans ton code PHP
```

PHP

```
<?php
// Toujours inclure ceci en premier dans ton fichier principal
require_once __DIR__ . '/vendor/autoload.php';

use Monolog\Logger;
use Monolog\Handler\StreamHandler;

// Créer un logger
$log = new Logger('wallet-app');
$log->pushHandler(new StreamHandler('logs/app.log', Logger::DEBUG));

// Utiliser le logger
```

```
$log->info('Nouveau wallet cree', ['telephone' => '77 000 00 00']);  
$log->error('Solde insuffisant', ['montant' => 5000, 'solde' => 2000]);  
?>
```

■ Le dossier vendor/ est cree automatiquement par Composer. Il contient tous les packages installes. IMPORTANT : ne jamais le versionner sur Git ! Ajoute vendor/ dans ton .gitignore. Un simple composer install le recreera.

7

Autoloading — Charger ses fichiers automatiquement

L'autoloading est l'une des fonctionnalités les plus puissantes de Composer. Il génère automatiquement un système qui charge tes classes PHP sans que tu aies à écrire des centaines de `require()` ou `include()`.

■ SANS Autoloading

```
<?php

require 'src/Wallet.php';

require 'src/User.php';

require 'src/Transaction.php';

require 'src/helpers/Format.php';

// ... pour chaque fichier

// Oublier un = erreur fatale
```

✓ AVEC Autoloading

```
<?php

require 'vendor/autoload.php';

// C'est tout !

// Toutes tes classes

// se chargent automatiquement

// quand tu les utilises
```

Configurer l'autoloading PSR-4

PSR-4 est le standard recommandé. Il fait correspondre un namespace PHP à un dossier.

JSON

```
// Dans composer.json

{
  "autoload": {
    "psr-4": {
      "App\\": "src/",
      "App\\Models\\": "src/models/",
      "App\\Services\\": "src/services/"
    }
  }
}
```

Ce que ça signifie concrètement :

Namespace dans le code

Fichier sur le disque

App\Wallet	src/Wallet.php
App\Models\User	src/models/User.php
App\Services\WalletService	src/services/WalletService.php

BASH

```
# Apres avoir modifie composer.json, regenerer l'autoloader
composer dump-autoload

# Version optimisee pour la production
composer dump-autoload --optimize
```

8

Composer dans un Projet PHP Reel

Voici comment integrer Composer dans un projet comme ton Wallet Manager.

Structure de fichiers recommandee

TEXT

```
wallet-manager/  
|-- src/  
| |-- Wallet.php <- namespace App  
| |-- User.php <- namespace App  
| |-- Transaction.php <- namespace App  
| |-- helpers/  
| |-- Format.php <- namespace App\Helpers  
|-- vendor/ <- GENERE par Composer (ne pas toucher)  
|-- logs/  
|-- .env <- Variables d'environnement  
|-- .gitignore <- Contient : vendor/ et .env  
|-- composer.json <- Configuration Composer  
|-- composer.lock <- Versions exactes installees  
|-- index.php <- Point d'entree
```

Le fichier composer.lock

✓ composer.lock est genere automatiquement. Il contient les versions EXACTES de chaque package installe. Contrairement a vendor/, il doit etre versionne sur Git. Quand un collegue clone ton projet et fait composer install, il obtient exactement les memes versions.

Le .gitignore pour un projet Composer

TEXT

```
# .gitignore  
vendor/  
.env  
*.log  
logs/
```

Packages utiles pour un Wallet Manager

Package	Utilite	Commande
vlucas/phpdotenv	Charger les variables .env	<code>composer require vlucas/phpdotenv</code>
monolog/monolog	Journalisation (logs)	<code>composer require monolog/monolog</code>
phpunit/phpunit	Tests unitaires	<code>composer require --dev phpunit/phpunit</code>
ramsey/uuid	Generer des IDs uniques	<code>composer require ramsey/uuid</code>

9

Bonnes Pratiques & Erreurs Courantes

✓ Bonnes pratiques	✗ Erreurs a éviter
Toujours versionner composer.lock	Versionner le dossier vendor/ sur Git
Ajouter vendor/ dans .gitignore	Modifier manuellement les fichiers dans vendor/
Specifiser une version PHP minimale dans require	Utiliser * comme version (trop permissif)
Separer require et require-dev	Installer des packages non maintenus (> 2 ans)
Lire le README du package avant installation	Oublier composer dump-autoload apres modif
Verifier le nombre de downloads (popularite)	Mettre .env sur Git (donnees sensibles !)

Workflow complet sur un nouveau projet

BASH

```
# 1. Creer le dossier et initialiser Composer
mkdir wallet-manager && cd wallet-manager
composer init

# 2. Installer les packages necessaires
composer require vlucas/phpdotenv
composer require monolog/monolog
composer require --dev phpunit/phpunit

# 3. Configurer l'autoloading dans composer.json
# (modifier le fichier pour ajouter la section autoload)

# 4. Regenerer l'autoloader
composer dump-autoload

# 5. Creer le point d'entree
# Dans index.php : require 'vendor/autoload.php';

# 6. Initialiser Git
git init
echo 'vendor/' > .gitignore
```

```
echo '.env' >> .gitignore  
git add .  
git commit -m 'feat: initialisation projet avec Composer'
```

★ Quand tu clones un projet existant qui utilise Composer, la première chose à faire est toujours : `composer install`. Cette commande lit le `composer.lock` et installe exactement les bonnes versions. Ne fais jamais `composer update` sur un projet clone sans avoir compris les changements.

Composer + Packagist = la combinaison gagnante pour tout projet PHP professionnel. Une fois que tu maîtrises ces outils, tu ne pourras plus t'en passer. Bonne construction de tes projets, Khadija !